UNIVERSITI TEKNOLOGI PETRONAS

TEB2103: MODELLING AND SIMULATION

SEPTEMBER 2026 SEMESTER

USV MARINE PLASTIC COLLECTION SIMULATION PROJECT REPORT

No.	Student name	Student ID	Programme
1	Ekhwan Danial Bin Ezarisma Azni	22010283	CS
2	Hannah Atiqah Binti Zulhazame	22010028	CS
3	Syahirah Binti Mohd Zaki	22010391	CS
4	Rasha Fera Binti Rizalman	22010597	CS

- Source code: rocketshipz/USV-lite-simulation on GitHub (commit c3d0bbc, `main.py`)

Contents

1. Introduction 3
   1.1 Background: plastic waste in rivers and seas 3
   1.2 Autonomous surface vehicles for cleanup 3
   1.3 Agent-based modelling and simulation 4
   1.4 Problem statement 4
2. Objectives and scope 5
   2.1 Objectives 5
   2.2 Scenario coverage 5
3. Model design 6
   3.1 Environment 6
   3.2 Agents 6
   3.3 Movement and load drag 7
   3.4 Parameters 7
4. Implementation 9
   4.1 Simulation loop 9
   4.2 Key logic: the hand-off 10
   4.3 How to run 10
5. Verification and validation 11
   5.1 Method 11
   5.2 Verification checks 11
   5.3 Validation 11
6. Experiments and results 12
   6.1 Baseline (Scenario 1) 12
   6.2 Sensitivity analysis 12
7. Discussion 15
   7.1 Agent-based behaviours shown 15
   7.2 What the results mean 15
   7.3 Limitations 16
8. Conclusion and future work 17
   8.1 Next steps 17

References 18
Appendix A: Full experiment results 19
Appendix B: Source code 21

1. Introduction

This project builds an agent-based simulation of two cooperating Unmanned Surface Vehicles (USVs) that find, collect and deliver floating plastic waste. In the baseline setup, the pair cleared 15 plastic items in an average of 629 time steps, and all 100 randomised runs finished.

1.1 Background: plastic waste in rivers and seas

Plastic is cheap, light and durable. These same properties mean that plastic waste can stay in the environment for decades once it escapes collection and recycling. Much of it ends up in water. The United Nations Environment Programme (UNEP) estimates that about 11 million tonnes of plastic pollution enter the ocean every year, and that up to 2.7 million tonnes of this is carried there by rivers (UNEP, n.d.).

Floating plastic causes harm in several ways. Animals can become entangled in it or mistake it for food. It also litters beaches, harbours and fishing areas. Over time, it breaks down into smaller fragments that are much harder to remove. Collecting plastic while it is still large and floating is therefore one of the most practical ways to stop it doing further damage.

Rivers are the main route that plastic takes from land to sea. Meijer et al. (2021) estimate that rivers carry 0.8 to 2.7 million tonnes of plastic into the ocean each year, and that more than 1,000 rivers account for 80% of this. Small rivers that flow through cities are among the largest sources. Malaysia is a relevant example. The same study estimates that Malaysian rivers carry about 73,000 tonnes of plastic to the sea each year, and it highlights the Klang River, which flows through Kuala Lumpur, as a small river that ranks highly. Rivers, harbours, lakes and sheltered coastal waters are therefore good places to intercept floating plastic before it reaches the open sea.

1.2 Autonomous surface vehicles for cleanup

Floating waste is usually collected by crewed boats using nets, or by workers along riverbanks and shores. This is slow and labour-intensive, it becomes expensive over large areas, and it exposes workers to polluted water.

A USV is a small boat that operates without anyone on board. It can be remote-controlled, or it can run on its own using GPS, sensors and onboard software. USVs built for waste collection already exist. For example, RanMarine's WasteShark is an autonomous surface vessel that can collect up to 500 kg of floating waste per day on an 8-hour battery. Its automated version follows preset routes using GPS, returns home at the end of its shift and uses onboard software to avoid obstacles (RanMarine, n.d.).

A single small vessel has clear limits. Its storage fills quickly, it must stop to unload, and it can only search the area close to it. One way to overcome these limits is to use a team of vessels with different

roles, where some collect waste and others transport it to shore. This keeps the collecting vessels working for longer. However, testing such a team with real boats is expensive and slow. A computer simulation makes it possible to try different designs and settings first, at very little cost.

1.3 Agent-based modelling and simulation

Agent-based modelling (ABM) represents a system as a collection of autonomous decision-making entities, called agents. Each agent has its own state and follows its own rules. The behaviour of the whole system emerges from how the agents act and interact (Macal & North, 2010).

ABM suits a multi-USV team well. Each vessel makes its own decisions based on what it senses, such as where the nearest plastic is or how full its storage is. The overall cleanup performance then depends on how well the vessels coordinate. This project uses Mesa, an open-source Python framework for agent-based modelling that provides ready-made building blocks for agents, models, space and scheduling (Kazil et al., 2020; ter Hoeven et al., 2025).

1.4 Problem statement

A single USV working alone is inefficient for three reasons:

- It can only search a limited area at a time.
- It must stop collecting and travel back to base whenever its storage is full.
- Currents, wind and obstacles can affect its movement, and it must avoid collisions.

This project proposes splitting the work between two roles. A **Search-and-Collect USV (SC-USV)** stays in the field and keeps collecting. A **Transport USV (T-USV)** carries each full load back to the base station. The simulation tests how well this division of labour works and what limits its performance.

2. Objectives and scope

This first version implements the proposal's Scenario 1 (normal collection) in full. Scenarios 2 and 3 will be added in the final version.

2.1 Objectives

1. Model the SC-USV and T-USV as autonomous agents with separate roles that communicate with each other.
2. Show goal seeking, sensor-based detection, area patrol and load hand-off between agents.
3. Measure cleanup performance: time to clear the area, transport trips, distance travelled and idle time.
4. Find out which design parameters affect performance most: storage capacity, sensor range, load drag, transport speed and amount of waste.

2.2 Scenario coverage

Proposed scenario	Behaviour it tests	Status in main.py
1. Normal plastic collection	Search, collect, hand-off, transport	Implemented
2. Strong current or wind	Moving targets, re-planning routes	Planned for final version (plastic stays still in this version)
3. Marine animal or obstacle	Obstacle detection, avoidance, waiting or rerouting	Planned for final version (no obstacles in this version)

This model also adds two features that were not in the proposal: plastic items have a weight, and that weight slows down the USV carrying it.

3. Model design

The model has three types of agent in a continuous 100 m × 100 m water area. Two of them, the SC-USV and the T-USV, are finite-state machines. They coordinate through each other's state and position.

3.1 Environment

- The water area is a bounded (non-wrapping) Mesa `ContinuousSpace` of 100 × 100 m.
- The base station is fixed at (10, 10).
- Time moves in discrete steps. At each step, every agent acts once, in a random order (`shuffle_do`).

3.2 Agents

Agent	Starts at	Key attributes	Role
Plastic	Random position, at least 5 m from the edges	weight 1.0–5.0 kg (random), `collected`	Passive target
SC-USV	(50, 50)	`capacity` (items), `current_weight`, `state`, `target_plastic`, `patrol_index`	Detects, collects, signals when full
T-USV	Base (10, 10)	`carried_weight`, `state`	Picks up the load, delivers it to base

SC-USV behaviour. While SEARCHING, the SC-USV checks for uncollected plastic within its sensor radius of 25 m. It locks onto the nearest item and moves straight towards it. When it reaches the item, it collects it. If nothing is in range, it moves through six fixed patrol waypoints. When it holds 3 items, it switches to WAITING and stops. When no plastic remains and its hold is empty, it becomes COMPLETED.

T-USV behaviour. The T-USV stays IDLE at base until the SC-USV's state is WAITING. It then switches to RETRIEVING and heads for the SC-USV's current position. When it arrives, it takes the whole load, which resets the SC-USV to SEARCHING. It then switches to RETURNING and heads to base. At base it unloads and becomes IDLE again.

Communication. The proposal describes agents sharing their location, capacity and task status. In the model, this is done by the T-USV reading the SC-USV's `state` and `pos` directly. Communication is assumed to be instant and reliable.

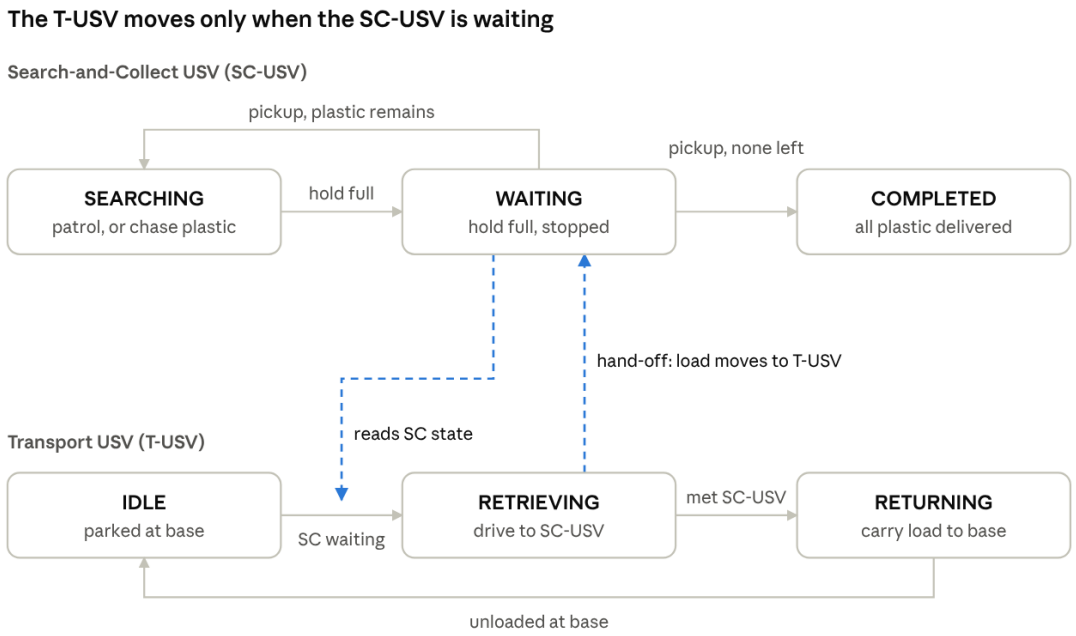


Figure 1: State machines in main.py · SC_USV_Agent and T_USV_Agent

The blue dashed lines are the only links between the two agents. The T-USV reads the SC-USV's state, and the hand-off moves the load across. "Hold full" also covers a partial load when no plastic is left.

3.3 Movement and load drag

Each step, a USV moves in a straight line towards its target by a distance equal to its speed. Speed decreases as the vessel carries more weight:

$$v = \max\left(v_{min}, \frac{v_0}{1 + kw}\right)$$

In this equation, v_0 is the unloaded speed, w is the carried weight in kg, k is the drag factor (0.08 per kg) and v_min is a lower limit of 0.3 so the vessel never stops completely. For example, a full SC-USV carrying 9 kg moves at 1.5 / 1.72 = 0.87 units per step, which is 58% of its unloaded speed.

3.4 Parameters

Parameter	Symbol in code	Baseline value
Area size	AREA_SIZE	100 m
Number of plastic items	NUM_PLASTICS	15
Plastic spawn margin	SPAWN_MARGIN	5 m from each edge

Parameter	Symbol in code	Baseline value
Plastic weight	PLASTIC_WEIGHT_MIN/MAX	1.0–5.0 kg, uniform
SC-USV storage	SC_CAPACITY	3 items
SC-USV unloaded speed	SPEED_SC	1.5 m/step
T-USV unloaded speed	SPEED_T	2.0 m/step
Drag factor	WEIGHT_SLOWDOWN_FACTOR	0.08 per kg
Minimum speed	MIN_SPEED	0.3 m/step
Sensor radius	SENSOR_RADIUS	25 m
Patrol waypoints	PATROL_WAYPOINTS	(20,20) → (20,80) → (50,80) → (50,20) → (80,20) → (80,80), repeating

4. Implementation

The model is written in Python 3 using Mesa 3 for agent-based modelling (ter Hoeven et al., 2025), NumPy for numerical work (Harris et al., 2020) and Matplotlib for the animation (Hunter, 2007). The code is in `main.py` on GitHub.

4.1 Simulation loop

1. `MarineCollectionModel.__init__` places the SC-USV, the T-USV and 15 plastic items with random positions and weights.
2. Each `model.step()` calls `step()` on every agent in a random order.
3. After each step, the script records a snapshot of positions, states, loads and speeds.
4. The run stops when no plastic is left, the T-USV is IDLE and the SC-USV is empty, or after 800 frames.
5. Matplotlib replays the snapshots as an animation. It shows the sensor circle, plastic weights and each USV's state, load and speed.

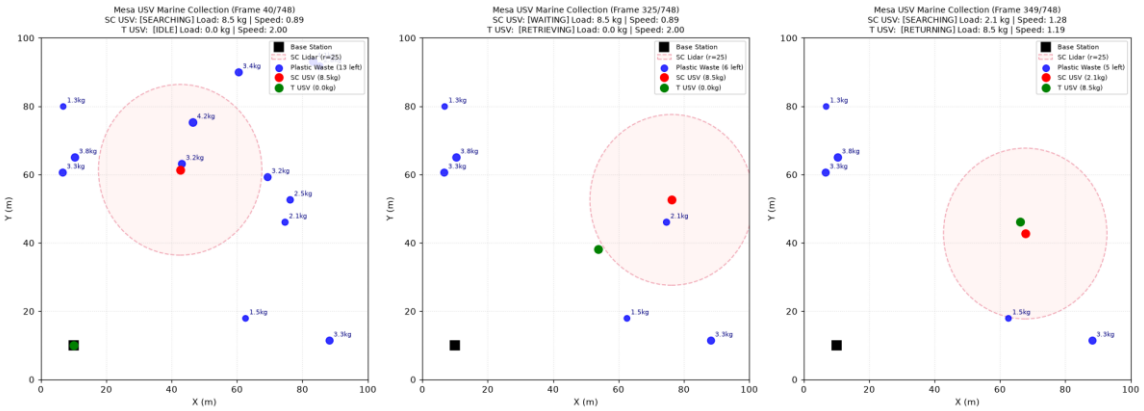


Figure 2: three frames from one run of the `main.py`, drawn with the script's own animation code. Frame 40: the SC-USV (red) is searching inside its 25 m sensor circle. Frame 325: its hold is full (8.5 kg), so it waits while the T-USV (green) drives out to it. Frame 349: after the hand-off, the SC-USV is searching again while the T-USV carries the 8.5 kg to base at a reduced speed of 1.19.

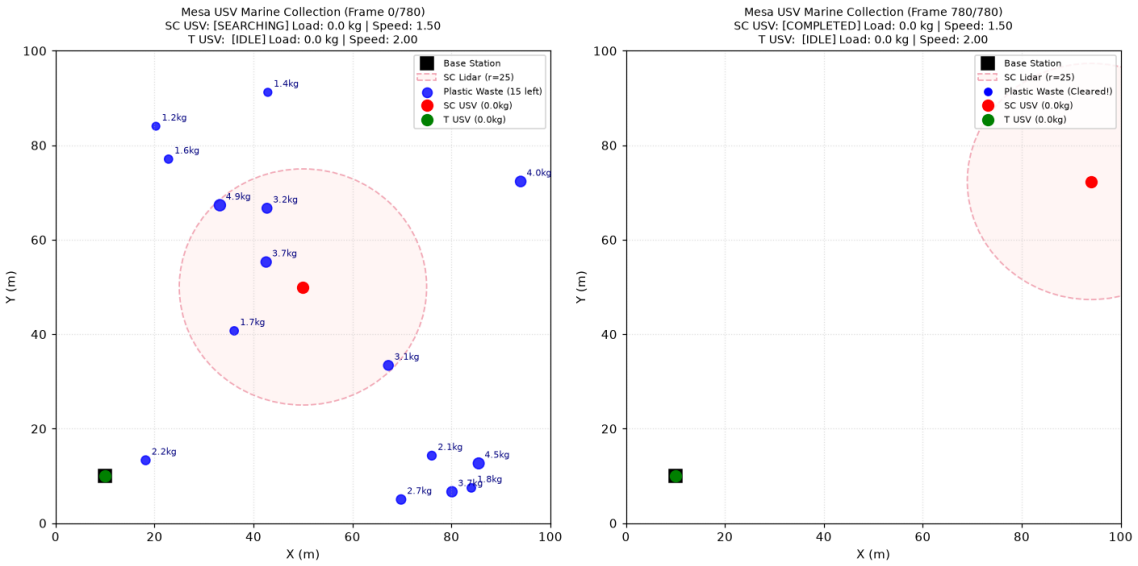


Figure 3: the start and end of one complete run of the main.py. Left, frame 0: 15 plastic items (1.2–4.9 kg) are scattered across the area, each at least 5 m from the edges. The SC-USV starts at the centre and the T-USV is at base. Right, final frame: every item has been delivered, the SC-USV is COMPLETED and the T-USV is IDLE at base with an empty hold.

4.2 Key logic: the hand-off

The T-USV's step() is where the two agents cooperate. This excerpt is from main.py:

```
if sc_agent.state == "WAITING" and self.state == "IDLE":
    self.state = "RETRIEVING"

if self.state == "RETRIEVING":
    reached = self.move_towards(np.array(sc_agent.pos), self.current_speed)
    if reached:
        self.carried_weight = sc_agent.current_weight   # take the load
        sc_agent.current_weight = 0.0
        sc_agent.capacity = 0
        sc_agent.state = "SEARCHING" if active_plastics else "COMPLETED"
        self.state = "RETURNING"
```

(Simplified: in the real code, active_plastics is computed just before the last if.)

4.3 How to run

```
pip install -r requirements.txt   # mesa>=3.0.0, numpy, matplotlib
python main.py
```

Mesa 3.5 also needs networkx, which is not listed in requirements.txt. We had to install it separately for the code to import.

5. Verification and validation

The model behaves as designed in every run. All verification checks passed.

5.1 Method

We ran the model classes from `main.py` (commit c3d0bbc) without the animation, 100 times per configuration. Each run used a different random seed (0–99). Runs were allowed up to 3,000 steps. For each run we recorded the following: completion step, transport trips, weight delivered, distance travelled by each USV, SC-USV waiting steps and SC-USV patrol steps.

5.2 Verification checks

Check	Expected	Result
Mass conservation	Weight delivered to base = weight generated	Held exactly in every completed run, across all 26 batches of 100 runs
Trip count	Trips = ⌈items ÷ capacity⌉	Matched in every configuration, e.g. 15 items ÷ 3 = 5 trips; 15 ÷ 4 → 4 trips
State transitions	Traced step by step for seed 0	SEARCHING → WAITING → hand-off → SEARCHING, ending in COMPLETED, as designed
Direction of effects	More drag or more items → slower; faster T-USV or larger sensor → faster	All held (Section 6)

5.3 Validation

The model has not been validated against real-world data. All speeds, weights and distances are illustrative, and "m/step" is an assumed unit. What we can show is face validity: in the animation, the agents behave as the proposal describes, and the effects of changing each parameter point in the expected direction.

6. Experiments and results

The SC-USV spends 31% of each run idle while it waits for the T-USV. Every run finishes as long as the sensor reaches at least 20 m. Extra storage stops saving time beyond 3 items.

6.1 Baseline (Scenario 1)

Measure (100 runs, baseline parameters)	Result
Runs that cleared all plastic	100 of 100
Steps to finish, mean ± SD	629 ± 95 (range 415–847)
Transport trips	5
Distance travelled, SC-USV / T-USV	425 m / 641 m
SC-USV steps spent waiting for pickup	191 (31% of the run)
SC-USV steps spent patrolling with nothing in range	175 (28% of the run)

The SC-USV is busy collecting for only about 40% of each run. The rest of the time is split between waiting for the T-USV and searching an empty area. The T-USV is the bottleneck. Each trip goes from base to the SC-USV and back, and on the return it carries a heavy load at reduced speed. If the SC-USV fills up again before the T-USV has unloaded, the SC-USV has to stop and wait. In the seed-0 trace, this happened at step 234, and the SC-USV waited 105 steps while the T-USV finished its delivery and drove back out.

6.2 Sensitivity analysis

Each experiment changed one parameter and kept the others at baseline. Each setting was run 100 times.

Parameter changed	Values tested	Mean steps to finish	Change vs baseline (629)
Load drag factor (per kg)	0 · 0.04 · 0.08 · 0.16	505 · 566 · 629 · 763	−20% to +21%
T-USV speed (m/step)	1.5 · 2.0 · 3.0	743 · 629 · 528	+18% to −16%
Number of plastic items	5 · 10 · 15 · 20 · 30	357 · 541 · 629 · 793 · 1,038	−43% to +65%

Without load drag, runs finish 20% faster. Increasing the T-USV's speed from 2 to 3 m/step cuts SC-USV waiting time from 191 to 116 steps. Run length grows steadily with the number of items, and the extra time is mostly waiting: with 30 items, the SC-USV waits 486 steps.

Storage capacity. Increasing capacity from 1 to 3 items cuts completion time from 1,126 to 629 steps, because there are fewer trips. From 3 to 6 items, completion time barely changes (613–638 steps), and at 8 items it rises to 724. A heavier hold slows the SC-USV, and the T-USV returns to base more slowly with a bigger load.

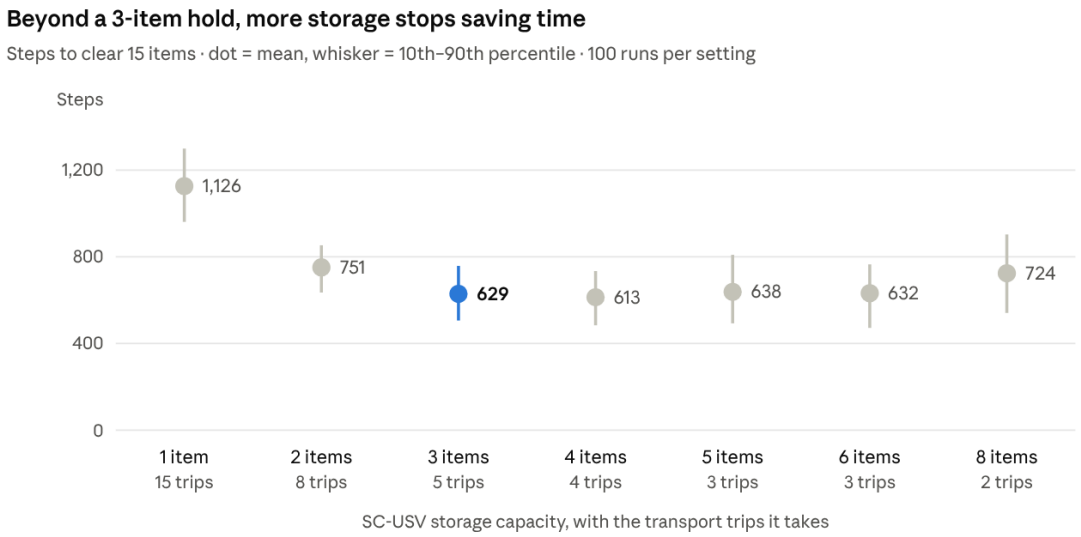


Figure 4: Batch runs of main.py · 100 seeded runs per capacity, 15 items, other parameters at baseline

Sensor range. The patrol route was designed around a 25 m radius. With a smaller radius, gaps open up between the patrol lines, and plastic in those gaps is never found.

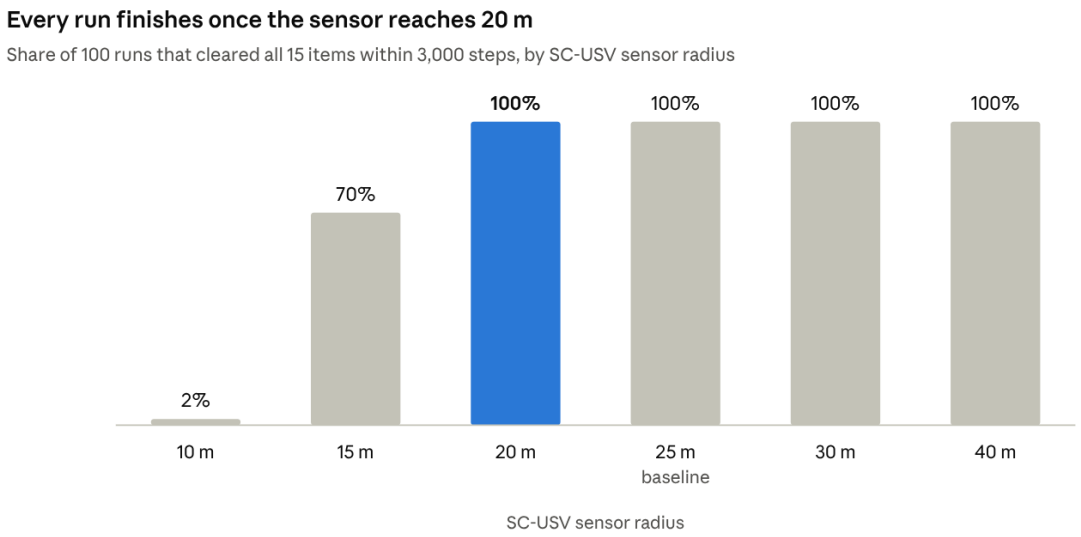


Figure 5: Batch runs of main.py · 100 seeded runs per radius, max 3,000 steps

At 10 m, only 2 runs finished, and the others left 1–8 items uncollected. At 15 m, 30 runs still got stuck. From 20 m upwards, every run finished.

7. Discussion

Splitting the roles works: the SC-USV never leaves the field to unload. However, a single T-USV cannot keep up.

7.1 Agent-based behaviours shown

Behaviour from the proposal	Shown?	Where in the model
Goal seeking	Yes	Both USVs move in a straight line towards the target plastic, the SC-USV or the base
Role specialisation	Yes	Separate SC-USV and T-USV classes with different state machines
Communication	Partly	The T-USV reads the SC-USV's state and position. No messages, delays or failures are modelled
Detection	Yes	25 m sensor radius, plus a patrol when nothing is in range
Adaptation	Partly	Speed drops with load. Targets do not move, so there is no re-planning
Obstacle avoidance	No	Planned for the final version (Scenario 3)

7.2 What the results mean

- **The T-USV is the bottleneck.** The SC-USV is idle for 31% of each run. A faster T-USV, or a second one, would reduce this directly: going from 2 to 3 m/step cut completion time by 16%.
- **Capacity has an optimum.** Too small a hold means many trips. Too large a hold makes the SC-USV slow and heavy. With the current drag factor, the gains stop at about 3 items, and holds of 3 to 6 items perform about the same. The baseline of 3 is a sound choice.
- **Coverage matters more than speed.** The patrol route is fixed and was designed around a 25 m sensor. With a sensor under 20 m, plastic between the patrol lines is still never found.

7.3 Limitations

- Only one SC-USV and one T-USV. The proposal describes a fleet.
- No currents, wind or moving plastic (Scenario 2), and no obstacles or animals (Scenario 3), in this version.
- Plastic is always detected and collected without fail, and transfers happen instantly when the T-USV arrives.
- No fuel or battery limits, and no collisions between the two USVs.
- Parameter values are illustrative and are not based on real USV specifications.

8. Conclusion and future work

We built a Mesa agent-based model of a two-role USV team. In normal conditions, it cleared 15 floating plastic items in about 629 steps, and all 100 test runs finished. Mass was conserved in every run, and trip counts matched the expected values. In the animation, the agents show goal seeking, sensor-based detection, patrol and a working hand-off.

8.1 Next steps

1. **Scenario 2 (final version):** add a current or wind effect that moves each plastic item every step. The SC-USV would update its target every step, and the T-USV would follow the SC-USV as it moves.
2. **Scenario 3 (final version):** add fixed obstacles and moving marine animals, each with a safety distance. The USVs would steer around them, and would wait or choose a different target when an animal stays near the plastic.
3. **Animation length (final version):** increase MAX_FRAMES in main.py from 800 so the animation always shows the full run. In testing, 2 of 100 runs needed more than 800 steps, so their animation stopped before the last delivery.
4. **Larger fleet:** use several SC-USVs that share the locations of detected plastic, so two vessels do not chase the same item, supported by one or more T-USVs.
5. **Code improvements:** add networkx to requirements.txt, and use Mesa's DataCollector to record the measurements used in this report.

References

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kazil, J., Masad, D., & Crooks, A. (2020). Utilizing Python for agent-based modeling: The Mesa framework. In *Social, Cultural, and Behavioral Modeling: SBP-BRiMS 2020* (pp. 308–317). Springer. https://doi.org/10.1007/978-3-030-61255-9_30

Macal, C. M., & North, M. J. (2010). Tutorial on agent-based modelling and simulation. *Journal of Simulation, 4*(3), 151–162. https://doi.org/10.1057/jos.2010.3

Meijer, L. J. J., van Emmerik, T., van der Ent, R., Schmidt, C., & Lebreton, L. (2021). More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean. *Science Advances, 7*(18), eaaz5803. https://doi.org/10.1126/sciadv.aaz5803

Project group. (2026). *USV marine plastic collection simulation* [Unpublished project proposal]. TEB2103 Modelling and Simulation, September 2026 semester.

RanMarine. (n.d.). *This aquatic robot is making a meal out of marine waste*. Retrieved October 13, 2026, from https://www.ranmarine.io/news/aquatic-robot-is-making-a-meal-out-of-marine-waste-2

rocketshipz. (2026). *USV-lite-simulation* (commit c3d0bbc) [Source code]. GitHub. https://github.com/rocketshipz/USV-lite-simulation

ter Hoeven, E., Kwakkel, J., Hess, V., Pike, T., Wang, B., rht, & Kazil, J. (2025). Mesa 3: Agent-based modeling with Python in 2025. *Journal of Open Source Software, 10*(107), 7668. https://doi.org/10.21105/joss.07668

United Nations Environment Programme. (n.d.). *Why we need to fix the plastic pollution problem*. Retrieved October 13, 2026, from https://www.unep.org/news-and-stories/story/why-we-need-fix-plastic-pollution-problem

Appendix A: Full experiment results

Each row is a batch of 100 runs (random seeds 0–99, up to 3,000 steps each) on the code (commit c3d0bbc). Only one parameter was changed per batch; all others were kept at their baseline values. Averages in the step, trip, waiting, patrol and distance columns are taken over the runs that finished.

Parameter changed	Value	Runs finished (of 100)	Steps, mean ± SD	Steps, 10th–90th percentile	Trips	SC-USV waiting steps	SC-USV patrol steps	SC-USV distance (m)	T-USV distance (m)
None (baseline)	–	100	629 ± 95	506–758	5	191	175	425	641
SC-USV storage	1 item	100	1,126 ± 127	961–1,299	15	785	136	425	1,841
SC-USV storage	2 items	100	751 ± 91	635–853	8	376	153	425	1,020
SC-USV storage	3 items (baseline)	100	629 ± 95	506–758	5	191	175	425	641
SC-USV storage	4 items	100	613 ± 96	484–734	4	150	185	425	530
SC-USV storage	5 items	100	638 ± 124	493–809	3	103	212	425	400
SC-USV storage	6 items	100	632 ± 114	472–765	3	117	205	425	417
SC-USV storage	8 items	100	724 ± 149	541–903	2	71	259	425	285
Sensor radius	10 m	2	644 ± 66	592–697	5	182	294	432	655
Sensor radius	15 m	70	649 ± 119	544–861	5	181	250	454	631
Sensor radius	20 m	100	633 ± 80	538–729	5	184	200	436	639
Sensor radius	25 m (baseline)	100	629 ± 95	506–758	5	191	175	425	641
Sensor radius	30 m	100	610 ± 101	484–751	5	195	144	406	635
Sensor radius	40 m	100	563 ± 99	462–726	5	198	80	351	628
Drag factor	0 per kg	100	505 ± 72	414–598	5	169	142	425	641

Parameter changed	Value	Runs finished (of 100)	Steps, mean ± SD	Steps, 10th–90th percentile	Trips	SC-USV waiting steps	SC-USV patrol steps	SC-USV distance (m)	T-USV distance (m)
Drag factor	0.04 per kg	100	566 ± 83	457–676	5	179	158	425	641
Drag factor	0.08 per kg (baseline)	100	629 ± 95	506–758	5	191	175	425	641
Drag factor	0.16 per kg	100	763 ± 121	609–942	5	223	208	425	641
Plastic items	5	100	357 ± 86	226–450	2	75	146	269	282
Plastic items	10	100	541 ± 90	431–669	4	178	170	371	555
Plastic items	15 (baseline)	100	629 ± 95	506–758	5	191	175	425	641
Plastic items	20	100	793 ± 84	681–892	7	307	188	500	906
Plastic items	30	100	1,038 ± 114	912–1,203	10	486	181	561	1,260
T-USV speed	1.5 m/step	100	743 ± 101	609–883	5	281	175	425	641
T-USV speed	2 m/step (baseline)	100	629 ± 95	506–758	5	191	175	425	641
T-USV speed	3 m/step	100	528 ± 90	406–645	5	116	174	425	641

Appendix B: Source code

The complete simulation is the file `main.py` in the project repository at commit c3d0bbc: github.com/rocketshipz/USV-lite-simulation. The full listing follows.

```
import mesa
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation


# Simulation Parameters
AREA_SIZE = 100.0
NUM_PLASTICS = 15
SPAWN_MARGIN = 5.0  # Margin from edges to spawn plastics
SC_CAPACITY = 3
SPEED_SC = 1.5
SPEED_T = 2.0
BASE_STATION = np.array([10.0, 10.0])


# Trash Weight & Movement Parameters
PLASTIC_WEIGHT_MIN = 1.0  # kg
PLASTIC_WEIGHT_MAX = 5.0  # kg
WEIGHT_SLOWDOWN_FACTOR  = 0.08  # Drag factor: higher weight -> slower movement
MIN_SPEED = 0.3  # Minimum drone speed floor to avoid stalling


# Lidar / Sensor Parameters
SENSOR_RADIUS = 25.0  # SC USV detection radius


# Search Patrol Waypoints for systematic area coverage when no trash is detected
PATROL_WAYPOINTS  = [
    np.array([20.0, 20.0]),
    np.array([20.0, 80.0]),
    np.array([50.0, 80.0]),
    np.array([50.0, 20.0]),
    np.array([80.0, 20.0]),
    np.array([80.0, 80.0]),
]



class PlasticAgent(mesa.Agent):
    def __init__(self, model, weight=None):
        super().__init__(model)
        self.collected = False
        if weight is None:
            self.weight = round(float(np.random.uniform(PLASTIC_WEIGHT_MIN, PLASTIC_WEIGHT_MAX)), 1)
        else:
```

```
        self.weight = float(weight)



class SC_USV_Agent(mesa.Agent):
    def __init__(self, model):
        super().__init__(model)
        self.capacity = 0
        self.current_weight = 0.0
        self.state = "SEARCHING"  # SEARCHING, WAITING, COMPLETED
        self.target_plastic = None
        self.patrol_index = 0

    @property
    def current_speed(self):
        # Drone moves slower the more trash weight it carries
        return max(MIN_SPEED, SPEED_SC / (1.0 + WEIGHT_SLOWDOWN_FACTOR * self.current_weight))

    def move_towards(self, target_pos, speed):
        current_pos = np.array(self.pos)
        direction = target_pos - current_pos
        distance = np.linalg.norm(direction)
        if distance < speed:
            self.model.grid.move_agent(self, tuple(target_pos))
            return True
        else:
            new_pos = current_pos + (direction / distance) * speed
            self.model.grid.move_agent(self, tuple(new_pos))
            return False

    def step(self):
        if self.state == "SEARCHING":
            active_plastics = [
                a for a in self.model.agents
                if isinstance(a, PlasticAgent) and not a.collected
            ]

            # If all plastics have been collected
            if not active_plastics:
                if self.capacity > 0:
                    self.state = "WAITING"
                else:
                    self.state = "COMPLETED"
                return

            # Detect plastics using Lidar / Radius sensor (only within SENSOR_RADIUS)
            detected_plastics = [
                p for p in active_plastics
                if np.linalg.norm(np.array(p.pos) - np.array(self.pos)) <= SENSOR_RADIUS
            ]
```

```
        # Reset target if it was already collected
        if self.target_plastic is not None and self.target_plastic.collected:
            self.target_plastic = None

        # If no target currently locked, lock onto the closest detected trash in sensor range
        if self.target_plastic is None and detected_plastics:
            distances = [
                np.linalg.norm(np.array(p.pos) - np.array(self.pos))
                for p in detected_plastics
            ]
            self.target_plastic = detected_plastics[np.argmin(distances)]

        if self.target_plastic is not None:
            reached = self.move_towards(np.array(self.target_plastic.pos), self.current_speed)
            if reached:
                self.target_plastic.collected = True
                self.model.grid.remove_agent(self.target_plastic)
                self.capacity += 1
                self.current_weight += self.target_plastic.weight
                self.target_plastic = None
                if self.capacity >= SC_CAPACITY:
                    self.state = "WAITING"
        else:
            # No trash detected within lidar radius -> patrol the area to find trash
            target_wp = PATROL_WAYPOINTS[self.patrol_index]
            reached = self.move_towards(target_wp, self.current_speed)
            if reached:
                self.patrol_index = (self.patrol_index + 1) % len(PATROL_WAYPOINTS)



class T_USV_Agent(mesa.Agent):
    def __init__(self, model):
        super().__init__(model)
        self.state = "IDLE"  # IDLE, RETRIEVING, RETURNING
        self.carried_weight = 0.0

    @property
    def current_speed(self):
        # Transport drone also slows down when carrying collected trash back to base
        return max(MIN_SPEED, SPEED_T / (1.0 + WEIGHT_SLOWDOWN_FACTOR  * self.carried_weight))

    def move_towards(self, target_pos, speed):
        current_pos = np.array(self.pos)
        direction = target_pos - current_pos
        distance = np.linalg.norm(direction)
        if distance < speed:
            self.model.grid.move_agent(self, tuple(target_pos))
            return True
```

```
        else:
            new_pos = current_pos + (direction / distance) * speed
            self.model.grid.move_agent(self, tuple(new_pos))
            return False

    def step(self):
        sc_agent = self.model.sc_agent
        if sc_agent.state == "WAITING" and self.state == "IDLE":
            self.state = "RETRIEVING"

        if self.state == "RETRIEVING":
            reached = self.move_towards(np.array(sc_agent.pos), self.current_speed)
            if reached:
                # Transfer collected trash load from SC drone to T drone
                self.carried_weight = sc_agent.current_weight
                sc_agent.current_weight = 0.0
                sc_agent.capacity = 0

                active_plastics = [
                    a for a in self.model.agents
                    if isinstance(a, PlasticAgent) and not a.collected
                ]
                if active_plastics:
                    sc_agent.state = "SEARCHING"
                else:
                    sc_agent.state = "COMPLETED"

                self.state = "RETURNING"

        elif self.state == "RETURNING":
            reached = self.move_towards(BASE_STATION, self.current_speed)
            if reached:
                # Unload trash at base station
                self.carried_weight = 0.0
                self.state = "IDLE"



class MarineCollectionModel(mesa.Model):
    def __init__(self, num_plastics=NUM_PLASTICS):
        super().__init__()
        self.grid = mesa.space.ContinuousSpace(AREA_SIZE, AREA_SIZE, False)

        # Add SC-USV
        self.sc_agent = SC_USV_Agent(self)
        self.grid.place_agent(self.sc_agent, (50.0, 50.0))

        # Add T-USV
        self.t_agent = T_USV_Agent(self)
```

```
        self.grid.place_agent(self.t_agent, tuple(BASE_STATION))

        # Add Plastics with random positions and weights
        for _ in range(num_plastics):
            p = PlasticAgent(self)
            pos = tuple(np.random.uniform(SPAWN_MARGIN, AREA_SIZE - SPAWN_MARGIN,  size=2))
            self.grid.place_agent(p, pos)

    def step(self):
        self.agents.shuffle_do("step")



# Start the simulation
# Instantiate Model
model = MarineCollectionModel()

# Prepare visualization data capture
history = []
MAX_FRAMES = 800

def capture_state():
    return {
        "sc_pos": np.copy(model.sc_agent.pos),
        "t_pos": np.copy(model.t_agent.pos),
        "sc_state": model.sc_agent.state,
        "t_state": model.t_agent.state,
        "sc_weight": model.sc_agent.current_weight,
        "sc_speed": model.sc_agent.current_speed,
        "t_weight": model.t_agent.carried_weight,
        "t_speed": model.t_agent.current_speed,
        "plastics": [
            {"pos": np.copy(a.pos), "weight": a.weight}
            for a in model.agents
            if isinstance(a, PlasticAgent) and not a.collected
        ]
    }

# Record initial frame
history.append(capture_state())

for frame in range(MAX_FRAMES):
    model.step()
    state = capture_state()
    history.append(state)

    # Check if all plastics are collected and T-USV has finished offloading at base
    if len(state["plastics"]) == 0 and state["t_state"] == "IDLE" and model.sc_agent.capacity == 0:
        # Linger for a few extra frames to show final completed state
```

```
        for _ in range(15):
            history.append(state)
        break

# Matplotlib Animation Setup
fig, ax = plt.subplots(figsize=(7, 7))

def update(frame):
    ax.clear()
    ax.set_xlim(0, AREA_SIZE)
    ax.set_ylim(0, AREA_SIZE)

    state_info = history[frame]
    ax.set_title(
        f"Mesa USV Marine Collection (Frame {frame}/{len(history)-1})\n"
        f"SC USV: [{state_info['sc_state']}] Load: {state_info['sc_weight']:.1f} kg | Speed: {state_info['sc_speed']:.2f}\n"
        f"T USV: [{state_info['t_state']}] Load: {state_info['t_weight']:.1f} kg | Speed: {state_info['t_speed']:.2f}",
        fontsize=9
    )

    # Plot Base Station
    ax.plot(BASE_STATION[0], BASE_STATION[1], 'ks', markersize=10, label="Base Station")

    # Draw Lidar Sensor Radius Circle around SC USV
    sc_pos = state_info["sc_pos"]
    sensor_circle = plt.Circle(
        (sc_pos[0], sc_pos[1]),
        SENSOR_RADIUS,
        edgecolor='crimson',
        facecolor='mistyrose',
        linestyle='--',
        alpha=0.35,
        linewidth=1.2,
        label=f"SC Lidar (r={int(SENSOR_RADIUS)})"
    )
    ax.add_patch(sensor_circle)

    # Plot Plastics
    plastics = state_info["plastics"]
    if plastics:
        px = [p["pos"][0] for p in plastics]
        py = [p["pos"][1] for p in plastics]
        sizes = [25 + p["weight"] * 12 for p in plastics]
        ax.scatter(px, py, c='blue', s=sizes, marker='o', alpha=0.8, label=f"Plastic Waste ({len(plastics)} left)")
        for p in plastics:
            ax.text(p["pos"][0] + 1.2, p["pos"][1] + 1.2, f"{p['weight']:.1f}kg", fontsize=7, color='navy')
    else:
        ax.plot([], [], 'bo', label="Plastic Waste (Cleared!)")
```

```
    # Plot Agents
    ax.plot(sc_pos[0], sc_pos[1], 'ro', markersize=9, label=f"SC USV ({state_info['sc_weight']:.1f}kg)")
    ax.plot(state_info["t_pos"][0], state_info["t_pos"][1], 'go', markersize=9, label=f"T USV ({state_info['t_weight']:.1f}kg)")


    ax.set_xlabel("X (m)")
    ax.set_ylabel("Y (m)")
    ax.grid(True, linestyle=':', alpha=0.4)
    ax.legend(loc="upper right", fontsize=7.5, framealpha=0.85)


ani = FuncAnimation(fig, update, frames=len(history), interval=35, repeat=False)
plt.show()
```